Convert timestamps into features (hour, day, season)

Project Report

Submitted in partial fulfilment of the requirements for the award of the

degree of

BACHELOR OF TECHNOLOGY

Submitted by:

Deepak Chand

section - 1CSE6

Roll No: 25SCS1003000999

Under the supervision of:

Mr. Ramvath Ganesh

Department of Computer Science and Engineering

Department of Computer Science and Engineering

IILM University

November 2025

Contents

Abstract

Introduction

Motivation—----------

Problem Statement—----------

Objective—----------

Scope of the Project—----------

Dataset

Historical Electricity Demand Data—----------

Timestamp-Based Features —----------

Lag Features & Rolling Statistics —----------

Limitations of the Dataset—----------

Dataset Preparation and Cleaning—----------

Data Justification—----------

METHODOLOGY

Data Collection & Preprocessing—----------

Feature Engineering —----------

Forecasting Model Development—----------

Anomaly Detection—----------

Optimization & Real-Time Decision Layer—----------

Implementation

Data Collection—----------

Data Preprocessing—----------

Feature Engineering—----------

Forecasting Model Development—----------

Anomaly Detection Module—----------

Optimization Engine—----------

Model Adaptation and Continuous Learning—----------

System Integration—----------

RESULT AND DISCUSSION

Feature Extraction Results—----------

Forecasting Model Performance—----------

Anomaly Detection Results—----------

Energy Distribution Optimization Results—----------

Overall System Evaluation—----------

Conclusion of Results—----------

Conclusion

Future Scope

ABSTRACT

The rapid growth of smart grids and renewable energy integration has made electricity demand increasingly dynamic, uncertain, and difficult to manage. This project presents an AI-powered system for **real-time energy demand forecasting, anomaly detection, and optimized energy distribution**. The solution focuses on converting timestamps into meaningful temporal features—such as hour, day, season, and calendar events—to improve model accuracy and responsiveness. Additional inputs such as historical load, weather data, and lagged demand values further enhance predictive performance.

Lightweight forecasting techniques, including ARIMA/SARIMA, exponential smoothing, and efficient neural networks, are employed to generate short-term demand predictions suitable for real-time deployment. Anomaly detection is performed through statistical residual analysis, Isolation Forest, and autoencoder-based reconstruction methods to identify sudden spikes, drops, or irregular grid behavior. Using these insights, an optimization module—based on approaches such as Model Predictive Control (MPC)—manages battery usage, demand response, and energy dispatch for improved stability and cost efficiency.

The system is adaptive, continuously retraining on incoming data to handle seasonal variations and shifts in consumption patterns. By integrating forecasting, anomaly detection, and optimization into a unified pipeline, this project enhances grid reliability, minimizes energy waste, and supports sustainable operation. The expected outcome is a scalable, real-time AI/ML framework that strengthens modern smart grid performance while enabling efficient renewable energy utilization.

1.Introduction

Motivation

Modern power grids are becoming increasingly complex due to the rapid integration of renewable energy sources such as solar and wind. While these sources support sustainability, they also introduce unpredictability and fluctuations in electricity supply. At the same time, consumer demand has become more dynamic, driven by urbanization, smart devices, and changing lifestyle patterns. This combination makes it difficult for traditional grid systems to maintain stability, reliability, and cost-effective energy distribution. Accurate forecasting and real-time anomaly detection are therefore essential for preventing overloads, blackouts, and energy wastage. Existing forecasting models often fail to adapt quickly to changing conditions, seasonal variations, or sudden demand spikes. Utilities need AI/MLdriven solutions that can understand temporal patterns—such as hour of day, day of week, and seasonal behavior—and make rapid and reliable predictions. This project is motivated by the need to build a lightweight, real-time, adaptive AI system that can: Predict short-term electricity demand with high accuracy, Detect abnormal usage patterns before they cause disruptions, and Optimize energy distribution for better grid efficiency. By leveraging AI and machine learning, the project aims to support smarter, more resilient, and eco-friendly energy systems that reduce operational costs and improve grid stability in the era of renewableenergyandsmartcities.

Problem Statement

Modern electricity grids experience highly variable and non-stationary demand due to growing renewable energy integration, unpredictable weather conditions, and changing consumer behavior. Traditional forecasting methods struggle to handle this volatility, leading to inefficiencies, higher operational costs, and increased risk of grid instability. Additionally, sudden anomalies such as unexpected spikes or drops in consumption often remain undetected until they cause disruptions. There is a critical need for a lightweight, real-time AI/ML

system capable of accurately forecasting short-term electricity demand, detecting anomalies promptly, and optimizing energy distribution. Such a system must efficiently extract temporal features—hour, day, season, and calendar events—and process historical and environmental data to ensure reliable and adaptive grid operations.

Objectives

Primary Objectives

To develop a real-time forecasting model that predicts short-term electricity demand using temporal features and historical load patterns.

To design an intelligent anomaly detection module capable of identifying sudden and abnormal variations in energy consumption.

To optimize energy distribution by using predictions and anomaly insights for better grid stability, battery management, and renewable integration.

Secondary Objectives

To preprocess and engineer key features such as hour, day, season, weather inputs, and past load values.

To build a lightweight ML/AI system suitable for fast, near-edge deployment with low latency.

To implement adaptive learning mechanisms that retrain/update models to handle seasonal changes and data drift. To evaluate system performance using metrics like RMSE, precision, recall, and anomaly detection accuracy.

Scope of the Project

Data Acquisition & Preprocessing

Collect historical and real-time electricity demand, weather data, and calendar information. Clean, filter, and handle missing or noisy data. Extract temporal features: hour, day, weekday/weekend, season, and special events.

Demand Forecasting System

Implement lightweight forecasting models (ARIMA/SARIMA, Holt-Winters, or efficient neural networks). Incorporate probabilistic forecasting for uncertainty estimation.

Anomaly Detection Module

Use statistical methods (Z-score), ML models (Isolation Forest, One-Class SVM), or autoencoderbased techniques. Detect sudden spikes, drops, or abnormal consumption patterns.

Energy Optimization Component

Use forecasting results to optimize battery usage, load shifting, demand response, and renewable dispatch. Implement techniques like Model Predictive Control (MPC) or rule-based optimization.

Adaptive Learning & Monitoring

Enable periodic retraining or online learning to address seasonal drift and new consumption behavior. Continuously evaluate performance and adjust thresholds or model parameters.

Deployment Scope

Design the system to be lightweight for near-edge/real-time use. Ensure scalability so it can be applied to small microgrids, smart homes, or large utility networks

Dataset

The dataset used for this project focuses on **electricity demand forecasting**, **temporal pattern extraction**, and **anomaly detection**. It combines multiple data sources that influence real-time power consumption.

1. Historical Electricity Demand Data

- Hourly or 15-minute interval energy consumption values
- Represents past load patterns of a region or utility
- Used to train forecasting and anomaly detection models

2. Timestamp-Based Features

Extracted directly from the date–time column:

- Hour of the day
- Day of the week
- Day of the month
- Month / Season
- Weekday vs. Weekend
- Holidays / Special events

These features help the model learn temporal trends.

3. Weather Data (Optional but Recommended)

- Temperature
- Humidity
- Wind speed
- Solar irradiance (important for renewable grids)

Weather variables strongly influence electricity demand.

4. Lag Features & Rolling Statistics

Created during preprocessing:

- Previous hour demand (lag-1)
- Previous day same hour demand (lag-24)
- Weekly lag (lag-168)
- Moving averages (3, 6, 12, 24 hours)

Improves model’s understanding of consumption pattern continuity.

5. Anomaly Labels (If Available)

- Spike events
- Demand drops
- Grid failures
- Sudden surges due to equipment malfunction

Limitations of the Dataset

- The dataset may have limited historical data, which makes it difficult to learn long-term seasonal patterns.
- Some values may be missing or noisy due to sensor errors or data collection issues.
- Weather information such as temperature and humidity may not be included, reducing forecasting accuracy.
- Anomaly labels may not be available, making supervised anomaly detection challenging.
- The dataset may only contain total electricity demand and not separate data for residential, industrial, or commercial users.
- The data might only be recorded at hourly intervals, limiting the model's ability to detect rapid changes.
- Information about grid operations, outages, or maintenance events is often missing.

- The dataset may represent only one region, making the model less generalizable to other locations.
- Renewable energy production data (solar/wind) may be absent, affecting smart grid forecasting.
- External influencing factors like holidays, population growth, and industrial activity may not be included

Dataset Preparation and Cleaning

1. Data Collection

- Gather historical electricity demand data from the grid or public datasets.
- Collect additional information such as weather data, calendar information, and timestamps.

2. Data Merging

- Combine demand data with weather and calendar data using the timestamp as the common key.
- Ensure all data sources are aligned to the same time interval (e.g., hourly).

3. Handling Missing Values

- Identify missing or null values in the dataset.
- Use methods like forward fill, backward fill, mean/median replacement, or interpolation to fill gaps.
- Remove rows only if they contain too much missing information.

4. Removing Outliers

- Detect abnormal spikes or drops that are caused by faulty sensors or errors.
- Use statistical methods (Z-score, IQR) to remove or correct unrealistic values.

5. Data Type Conversion

- Convert timestamp column into proper datetime format.

- Ensure numerical columns like demand, temperature, and humidity are in the correct numeric format.

6. Feature Extraction (from Timestamp)

- Extract hour, day, weekday/weekend, month, and season.
- Create binary flags for holidays or special days.

7. Creating Lag and Rolling Features

- Generate lag features such as previous hour demand (lag-1), previous day same hour (lag-24), and weekly lag (lag-168).
- Create rolling averages (e.g., 6-hour, 12-hour, 24-hour moving averages) to capture trends.

8. Scaling the Data (if needed)

- Apply normalization or standardization to features, especially before using ML models like neural networks.

9. Removing Duplicates

- Check for duplicate rows with the same timestamp and remove them to avoid bias.

10. Final Dataset Preparation

- Arrange the cleaned data in a structured format.
- Split the dataset into training, validation, and testing sets.
- Save the final dataset in CSV format for model training.

Data Justification

The dataset used in this project is justified because it captures the key factors that directly influence electricity demand and grid behavior. Historical demand data provides a strong foundation for understanding past consumption patterns, which is essential for accurate forecasting. Timestamp-based features such as hour, day, and season help the model learn daily and seasonal variations that commonly affect energy usage.

Weather data, including temperature and humidity, is highly relevant because electricity consumption often increases during extreme weather conditions. Including these variables improves the model’s ability to predict real-world demand changes. Lag features and rolling

averages are also important because electricity usage is strongly dependent on recent consumption trends.

Calendar information such as weekends and holidays adds contextual knowledge, allowing the model to adjust predictions for non-routine days. Together, these features create a comprehensive dataset that supports accurate forecasting, reliable anomaly detection, and effective energy optimization. The chosen data is relevant, practical, and directly supports the goals of building an AI-based smart energy management system.

METHODOLOGY

The methodology for the **AI-Powered Smart Energy Demand Forecasting & Optimization System** is organized into five major phases:

1. **Data Collection & Preprocessing**
2. **Feature Engineering (including timestamp feature extraction)**
3. **Forecasting Model Development**
4. **Anomaly Detection System**
5. **Optimization & Real-Time Adaptation**

Data Collection & Preprocessing

This phase focuses on gathering all relevant data and preparing it for machine learning.

1.1 Data Sources

- **Historical electricity demand data**
- **Real-time load readings from smart meters/sensors**
- **Weather parameters** (temperature, humidity, solar radiation, wind speed)
- **Calendar information** (holidays, weekends, seasons)
- **Renewable energy availability data** (solar/wind generation, if applicable)

1.2 Data Cleaning

- Handle missing values using interpolation or statistical imputation.
- Remove noise and outliers using smoothing filters.
- Convert timestamp formats into a unified datetime index.
- Validate data consistency (detect duplicated, corrupted, or misaligned entries).

1.3 Data Normalization

- Apply Min-Max scaling or Standardization for ML algorithms.
- Normalize weather and load ranges for stable model training.

Feature Engineering

The core of this project involves extracting meaningful patterns from timestamps and contextual variables.

2.1 Timestamp Feature Extraction

From the datetime column, the following features are generated:

Feature	Description
Hour	Identifies daily load cycle patterns.
Day of Week	Helps differentiate weekday vs weekend demand.
Month	Used for seasonal variations.
Season	Summer, Winter, Monsoon, etc.
Is_Weekend	Binary feature to capture behavioral changes.
Holiday Flag	Marks government holidays that affect consumption.

2.2 Lag & Rolling Features

- **Lagged values:** load(t−1), load(t−24), load(t−168)
- **Rolling statistics:** 3-hour / 6-hour / 24-hour moving averages
- **Load differences:** Δload(t) to capture dynamic changes

2.3 External Context Features

- Weather-based features (temperature, humidity, rainfall, etc.)
- Renewable availability (solar/wind output forecasts)

---

3. Forecasting Model Development

The aim is to construct a **lightweight and real-time model**.

3.1 Train-Test Split

- Sliding-window or walk-forward validation is used for time-series prediction.
- Ensures no future data leakage.

3.2 Baseline Models

- ARIMA / SARIMA
- Holt-Winters Exponential Smoothing

3.3 Machine Learning Models

- Random Forest Regressor
- Gradient Boosting Regressor (XGBoost, LightGBM)

3.4 Deep Learning Models (if resources allow)

- LSTM-based sequence model
- GRU
- Temporal Convolutional Networks (TCN)

3.5 Probabilistic Forecasting

To estimate uncertainty:

- Quantile Regression
- Masked Multi-Step Probabilistic Forecasting

3.6 Evaluation Metrics

- RMSE
- MAE
- MAPE

- Prediction Interval Coverage Probability (PICP)

Anomaly Detection

This module identifies abnormal patterns such as sudden spikes or unexplained drops.

4.1 Residual-Based Detection

- Forecast vs actual difference
- Statistical thresholds using Z-score or MAD

4.2 Machine Learning Approaches

- Isolation Forest
- One-Class SVM
- Local Outlier Factor

4.3 Deep Learning Approaches (optional)

- Autoencoder reconstruction error
- LSTM Autoencoder anomaly detection

Optimization & Real-Time Decision Layer

This phase uses the forecast + anomalies to make energy distribution decisions.

5.1 Optimization Algorithms

- Model Predictive Control (MPC)
- Linear/Quadratic programming
- Rule-based dispatch for renewables and storage

5.2 Real-Time Energy Management

- Battery charging/discharging
- Load shifting

- Demand-response automation
- Renewable integration optimization

5.3 Adaptive Learning

- Periodic retraining using new data
- Online learning for drift adaptation
- Dynamic threshold adjustment for anomaly detection

Implementation

The implementation of the *AI-Powered Smart Energy Demand Forecasting and Optimization System* was carried out in a structured, step-by-step manner. The workflow includes data preparation, feature engineering, model development, anomaly detection, and optimization logic. Each phase was designed to support real-time, lightweight, and adaptive energy management suitable for smart grid environments.

---

1. Data Collection

Historical and real-time data was gathered from multiple sources such as:

- Electricity demand/load data
- Weather data (temperature, humidity, cloud cover)
- Calendar-based information (hour, weekday, weekend, holidays, season)

This multi-source dataset allows the model to understand temporal, environmental, and behavioral patterns influencing demand.

2.Data Preprocessing

Collected raw data contained missing values, noise, and inconsistencies. The following preprocessing steps were applied:

- Handling missing values using interpolation and forward-fill
- Removing duplicate records
- Filtering out extreme outliers using statistical thresholds
- Converting timestamps into structured attributes (hour, day, month, weekday/weekend, season)
- Normalizing numerical features for model stability
- Creating lag-based features (previous hour/day demand) for time-series modeling

This ensures the dataset is clean, consistent, and ready for forecasting.

---

3. Feature Engineering

To help the model capture time-based patterns, several temporal features were extracted:

- **Hour of the day** – captures daily consumption patterns
- **Day of the week** – distinguishes weekdays vs. weekends
- **Month and season** – accounts for seasonal demand variations
- **Holiday flag** – detects special-day effects
- **Lag features** – past demand at t-1, t-6, t-24 hours
- **Rolling averages** – smoothed demand trends

These features improve forecasting accuracy and help detect anomalies more effectively.

4.Forecasting Model Development

A lightweight short-term forecasting model was developed to predict energy demand. The following algorithms were evaluated:

- ARIMA / SARIMA
- Holt-Winters exponential smoothing
- Gradient Boosted Trees (XGBoost/LightGBM)
- LSTM-based small neural networks (optional depending on deployment constraints)

The final selected model offered:

- Low latency
- High prediction accuracy
- Low computational cost suitable for near-edge deployment

The model outputs next-step or multi-step future demand predictions.

5.Anomaly Detection Module

To identify unusual grid behavior such as sudden spikes or drops, an anomaly detection module was implemented. It uses:

- **Forecast residual analysis** (difference between actual and predicted values)
- **Z-score based statistical detection**
- **Isolation Forest / One-Class SVM** for ML-based anomaly detection
- **Autoencoder reconstruction error** for pattern irregularities

This multi-method approach ensures robust detection of unexpected consumption patterns.

---

6. Optimization Engine

Using the predicted demand and anomaly signals, an optimization engine was implemented to assist grid operations. It performs:

- Battery storage scheduling
- Demand-response activation
- Renewable energy dispatch
- Load shifting to off-peak hours

Techniques such as **Model Predictive Control (MPC)** and rule-based decision logic were used to maintain grid stability while minimizing operational cost.

7.Model Adaptation and Continuous Learning

To ensure long-term reliability, continuous monitoring and updating mechanisms were integrated:

- Sliding window retraining
- Automatic threshold adjustment for anomaly detection
- Drift detection for seasonal changes
- Evaluation using RMSE, MAE, Precision, Recall, and F1-Score

This keeps the system adaptive to evolving consumption behavior.

---

8. System Integration

All modules—forecasting, anomaly detection, optimization—were integrated into a unified pipeline capable of real-time operation. The system was tested for:

- Scalability
- Latency
- Fault tolerance
- Accuracy under varying conditions

RESULT AND DISCUSSION

1. Feature Extraction Results

After converting timestamps into meaningful temporal features, the dataset gained several new signals such as **hour of day**, **day of week**, **month**, and **season**.
These features significantly improved the model's understanding of daily and seasonal consumption patterns.

Key Observations

- **Hour** captured peak-time usage patterns (morning & evening).
- **Day of week** revealed lower demand on weekends.
- **Season** highlighted winter and summer demand surges due to heating/cooling.
- These engineered features improved forecasting accuracy by providing richer contextual information.

. Forecasting Model Performance

Lightweight models such as **ARIMA/SARIMA**, **Holt-Winters**, and **small neural networks like LSTM** were evaluated for short-term load forecasting.

Performance Metrics

- Models achieved **lower RMSE and MAE** after adding temporal features.
- LSTM performed better for capturing long-term dependencies but required higher computation.
- SARIMA provided the best balance between **accuracy and real-time efficiency**.

Discussion

- Temporal features improved prediction stability.
- The model successfully learned recurring demand cycles (e.g., morning peaks).
- Lower latency models worked well for real-time grid environments.

Anomaly Detection Results

Anomaly detection was applied on residuals (actual – predicted values) using **Z-Score**, **Isolation Forest**, and **Autoencoders**.

Findings

- Isolation Forest detected abnormal sudden spikes effectively.
- Autoencoders identified rare patterns and irregular consumption behavior.
- Most anomalies occurred during festival days or unexpected weather changes.

Discussion

- Combining forecasting + anomaly detection provided strong grid-monitoring capability.
- Early anomaly alerts can prevent overloads or equipment failures.

Energy Distribution Optimization Results

Using demand forecasts and detected anomalies, optimization algorithms such as **Model Predictive Control (MPC)** improved energy flow decisions.

Key Optimization Outcomes

- Reduced energy wastage by scheduling battery charge/discharge intelligently.
- Lowered peak demand using demand-response strategy.
- Improved renewable integration by predicting available solar/wind input ahead of time.

Discussion

- Optimization stabilized grid behavior during high-demand hours.
- Forecast-driven battery operation reduced reliance on expensive backup generators.
- The system reacted adaptively to anomalies by reallocating energy resources.

Overall System Evaluation

The integration of forecasting, anomaly detection, and optimization created a **robust smart-grid AI system**.

System Strengths

- **High accuracy** in short-term load prediction.
- **Real-time responsiveness** suitable for edge-level deployment.
- **Low computational cost**, making the system scalable.
- **Adaptive learning** ensured performance stability across seasons.

Challenges

- Dataset quality (missing values, noise) impacted model behavior.
- Weather data variations increased uncertainty.
- Peak-time anomalies were harder to detect due to natural fluctuations.

Conclusion of Results

The system successfully:

- Extracted meaningful temporal features
- Improved forecasting accuracy using efficient AI/ML models
- Detected anomalies reliably
- Optimized smart-grid energy distribution

Overall, the results demonstrate that AI-powered forecasting and optimization can **significantly enhance grid efficiency, stability, and sustainability**.

Conclusion

The project successfully demonstrates how AI/ML techniques can enhance efficiency, stability, and decision-making in modern smart grid systems. By converting timestamps into meaningful temporal features (hour, day, season) and integrating them with historical load and contextual data (such as weather and events), the system provides accurate short-term demand forecasts. The implementation of anomaly detection further ensures early identification of abnormal consumption patterns, allowing proactive maintenance and improved grid resilience.

The proposed system also highlights the importance of lightweight and real-time models, making it suitable for near-edge deployment. Optimization logic—supported by forecasted demand and detected anomalies—enables more intelligent distribution of energy, better battery dispatch, load shifting, and smoother integration of renewable sources. Overall, the project shows that AI-driven forecasting and optimization can significantly reduce operational costs, improve energy reliability, and support sustainability goals in next-generation smart grids.

Future Scope

Future Enhancement Area	Description
1. Integration of Advanced Neural Models	Models like **Temporal Fusion Transformers (TFT)** and **Graph Neural Networks** can be incorporated to capture deeper temporal and spatial patterns across grid nodes.
2. Renewable Energy Supply Forecasting	Future systems can jointly forecast **solar, wind, and load demand**, enabling more balanced grid operation under renewable variability.
3. Full Smart-Grid Automation	Combining forecasting with real-time control (MPC, reinforcement learning) can automate energy dispatch, battery storage, and demand response.
4. Edge-AI Deployment	Light-weight TFLite or ONNX models can be deployed on IoT devices for **near-source forecasting**, reducing communication delays.
5. Digital Twins for Power Grids	A virtual replica of the grid can be developed to simulate failures, test optimization strategies, and improve operational reliability.
6. Incorporating EV Charging & Smart Homes	Growing adoption of electric vehicles and smart appliances introduces new patterns; including these loads can improve forecasting accuracy.

7. Online Learning & Continuous Adaptation	Implementing **self-updating models** will help handle seasonal drifts, consumer behavior changes, and unexpected events in real time.
8. Multi-Objective Optimization	Future versions can optimize not just cost, but also **emissions, grid stress, carbon footprint, and customer satisfaction**.
9. Grid-Level Security Anomaly Detection	AI models can be expanded to detect **cyber-attacks, false data injection, and system faults** for enhanced security.